ETS4 - A02 - Control de versiones - Git

En esta actividad procedemos a realizar distintos ejercicios en los cuales deberemos crear nuestro repositorio git local ty realizar una serie de acciones que nos permitan avanzar en el uso de git como sistema para el control de versiones. Además seguimos repasando trabajando con Javadoc en la documentación del código proporcionado.Se trata de ejercicios guiados..

INDICACIONES

Para la entrega de esta actividad realiza las distintas actividades en tu equipo, **realiza capturas de pantalla de los procesos más relevantes solicitados en cada apartado** y **elabora un documento donde vayas describiendo detalladamente las distintas actividades con fotos y texto**. Además el documento deberá:

- ➔ Seguir las numeraciones establecidas en las distintas actividades.
- ➔ Bibliografía (cuando sea necesaria)

OBJETIVO

Hacer uso correcto de git y de Javadoc e irnos habituando al desarrollo de código de forma profesional.

ENTREGA:

Una vez termines genera un documento PDF, guardarlo como ***A402-nombreapellidos*** y adjúntalo a la entrega.

Continúa en la siguiente página …

ACTIVIDAD: Control de versiones - Git

Ejercicio 1: Documentación y Control de Versiones con Git

El objetivo de esta actividad es el uso de la línea de comandos de Git y la correcta aplicación de etiquetas Javadoc en un entorno de desarrollo profesional como IntelliJ IDEA.

Requisitos:

La actividad se realizará íntegramente desde la terminal (Git Bash) de IntelliJ.

Enunciado:

Eres el nuevo desarrollador de la biblioteca del I.E.S. El Rincón. Tienes que documentar una clase básica y gestionar sus versiones mediante Git.

Fase 1: Inicialización y Primer Commit (Git)

1. Crea un nuevo proyecto en IntelliJ llamado **BibliotecaRincon**.
2. Abre la terminal de IntelliJ y asegúrate de seleccionar **Git Bash**.
3. Inicializa el repositorio localmente usando el comando `git init`.
4. Crea la clase **Libro.java** con el código que se proporciona al final de este documento.
5. Añade los archivos al área de preparación (*staging*) con `git add .`.
6. Realiza el primer `commit` con el mensaje: `"Initial commit: Estructura básica de la clase Libro"`.

Fase 2: Documentación Javadoc

Añade comentarios Javadoc a la clase **Libro.java** cumpliendo los siguientes requisitos:

1. **Clase**: Incluye descripción, `@author` (tu nombre), `@version 1.0` y `@since 2026`.
2. **Constructor**: Documenta los parámetros `@param titulo`, `@param autor` e `@param isbn`.
3. **Métodos Getter**: Documenta qué devuelven usando `@return`.
4. **Método prestar()**: Documenta que marca el libro como no disponible.

5. **Etiqueta Especial**: En la descripción de la clase, usa `{@link #prestar()}` para referenciar el método de préstamo.

Fase 3: Gestión de Ramas y Conflictos (Git)

1. Crea una nueva rama llamada **feat-documentacion** con el comando `git switch -c feat-documentacion`.
2. Realiza un commit en esta rama con los cambios de Javadoc con el siguiente mensaje:`"Añadida documentación Javadoc completa"`.
3. Vuelve a la rama **master** (`git switch master`).
4. En **master**, modifica la línea del `@author` en el código (pon "Administrador") y haz commit con el siguiente mensaje: `"Cambio de autor en master"`.
5. Intenta fusionar la rama de documentación: `git merge feat-documentacion`.
6. **Resuelve el conflicto** que aparecerá en el archivo **Libro.java**, elige la versión que tiene tu nombre, limpia las marcas de Git y finaliza el merge con un nuevo commit.

Fase 4: Versión Final y Generación

1. Crea una etiqueta de versión: `git tag -a v1.0 -m "Versión documentada y estable"`.
2. Muestra el historial gráfico en consola: `git log --oneline --graph --all`.
3. Genera la documentación HTML desde el menú **Tools > Generate JavaDoc...** y verifica que todo sea correcto.

Continúa en la siguiente página …

Código

El código es el siguiente.

```java
public class Libro {
    private String titulo;
    private String autor;
    private String isbn;
    private boolean disponible;

    public Libro(String titulo, String autor, String isbn) {
        this.titulo = titulo;
        this.autor = autor;
        this.isbn = isbn;
        this.disponible = true;
    }

    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }
    public void prestar() { this.disponible = false; }
}
```

Ejercicio 2: Flujo de Trabajo Profesional con Git y Javadoc

El objetivo de esta actividad es simular un entorno de desarrollo real donde el código evoluciona en distintas ramas, requiriendo una documentación técnica precisa y una gestión de versiones sin errores desde la terminal.

Requisitos:

La actividad se realizará íntegramente desde la terminal (Git Bash) de IntelliJ.

Enunciado:

Vas a trabajar en el núcleo de un sistema de reservas. El código base tiene errores de diseño (que ignoraremos de momento), pero necesita urgentemente ser documentado y versionado correctamente.

Fase 1: Configuración de Base (Consola Git)

1. Crea un proyecto llamado **HotelRincon**.
2. Inicializa Git, crea la clase **Reserva.java** (código abajo) y haz el primer commit en la rama **master**.
3. Crea una **etiqueta (tag)** llamada `v0.1-alpha`.

Fase 2: Desarrollo en Paralelo (Ramas)

Aquí es donde se complica. Debes trabajar en dos ramas distintas para simular dos tareas diferentes:

Tarea A: Rama docs-basica

1. Crea y salta a la rama **docs-basica**.
2. Documenta la clase y el constructor usando:
   - `@author`, `@version`, `@since`.
   - `@param` para todos los atributos.
   - Usa **etiquetas HTML** (`<ul>` y `<li>`) en la descripción de la clase para listar las reglas de reserva (ej: "Mínimo 1 noche", "Check-in a las 14:00").
3. Haz commit de estos cambios.

Tarea B: Rama docs-avanzada (Desde master)

1. Vuelve a la rama **master** y crea una nueva rama llamada **docs-avanzada**.
2. En esta rama, documenta el método `calcularPrecio(int noches, double tarifa)`:
   - Usa `@param`, `@return`.

- Usa `@throws IllegalArgumentException` explicando que no se aceptan noches negativas.
- Usa la etiqueta `{@code ...}` para poner un ejemplo de fórmula de cálculo: `{@code precio = noches * tarifa * 1.10}` (10% de IVA).

3. Haz commit.

Fase 3: Integración y Resolución de Conflictos

1. Vuelve a **master**.
2. Fusiona (`merge`) la rama **docs-basica**. (No debería haber conflicto).
3. Ahora, intenta fusionar **docs-avanzada**. **Aquí surgirá el conflicto** porque ambas ramas han tocado partes cercanas del archivo.
4. **Resolución Manual**: Abre el archivo, decide qué partes de la documentación de ambas ramas son válidas (deberías mantener ambas) y limpia las marcas de Git (`<<<<<<<`, `=======`, `>>>>>>>`).
5. Finaliza el proceso con `git add` y `git commit`.

Fase 4: Referencias Cruzadas y Validación

1. Crea una segunda clase llamada **Cliente.java** muy simple (nombre y DNI).
2. En **Reserva.java**, añade un método `asignarCliente(Cliente c)`.
3. Documenta este método usando `{@link Cliente}` para que en el HTML se pueda pinchar y navegar a la clase Cliente.
4. Genera el Javadoc final y comprueba que los enlaces (links) y las listas HTML funcionan.

Código

El código es el siguiente.

```java
import java.util.Date;

public class Reserva {
    private String idReserva;
    private String habitacion;
    private double precioBase;

    public Reserva(String idReserva, String habitacion, double precioBase) {
        this.idReserva = idReserva;
        this.habitacion = habitacion;
        this.precioBase = precioBase;
    }

    public double calcularPrecio(int noches, double tarifa) {
        if (noches < 0) throw new IllegalArgumentException();
        return noches * tarifa;
    }
}
```